Struct

- Elimizdeki mevcut tipleri kullanarak yeni tipler tanımlarız. Eğer bilgisayar seviyesinde olsaydık yani renkleri biz yaratsaydık kendimize özgü veri tipleri oluşturabilirdik ama yapamıyoruz. (C özelinde)
- X diye bir tip oluşturduğumuzda bu tipi yaparken temel tipleri (char, integer, float) kullanarak özelleştirir.
- Struct yapısı istenirse global alana istenirse herhangi bir fonksiyon içine tanımlanabilir.

```
struct yapı_tipi_adi{
    veri_tipi yapi_elemani1;
    veri_tipi yapi_elemani2;
    //.......;
};     //Noktalı virgüülü unutma, fonksiyon prototipi gibi düşün.
```

- Yeni bir tip oluşturduktan sonra bunları “riceb x;” gibi x variuble’sini riceb tipinde tanımlayabiliriz.

Kendi Ürettiğimiz Tipte Variable Atama

```
struct x{
    ....;
    ....;
    ....;
};

struct x y; //x türünde y değişkeni bu şekilde başına "struct" yazarak oluşturduk.;
```

Örnek:

```
struct ogrenci{
    char isim[15];
    int yas;
    float ortalama;
    char cinsiyet;
};

struct ogrenci kisi={"Ali Yılmaz",18,3.50,'E'};
```

```
struct konum{
    int x,y;
}n1;



printf("Lutfen noktanın x ve y koordinatlarini giriniz:\n);
scanf("%d %d",&n1.x,&n1.y);
--------------------------------------------------------------------------------
--------
Lutfen noktanın x ve y koordinatlarini giriniz:
50
60
//fin.
```

Struct Tiplerine Erişim

```
struct ogrenci{
    char isim[15];
    int yas;
    float ortalama;
    char cinsiyet;
};

struct ogrenci kisi,riceb={"Ali Yılmaz",18,3.50,'E'}; //kisi ve riceb ogrenci tipind
e.

kisi.isim[0]; //'A'
kisi.yas=20 //Yaş artık 20.
if(kisi.ortalama>2){
    komut;
    komut;
    komut;
}
```

```
struct ogrenci{
    char isim[15];
    int yas;
    float ortalama;
    char cinsiyet;
};

struct ogrenci kisi,riceb={"Ali Yılmaz",18,3.50,'E'};

kisi=riceb; //Ricebin struct verileri kisi değişkenine atandı.
```

Oluşturulan Tiplerde Pointer

- Pointerler gösterdikleri değişkenin tipinde olması gerektiği için eğer değişken bizim atadığımız bir tipteyse pointer de aynı tipte olmalıdır.

```
struct ogrenci{
    char isim[20];
    int yas;
    float ortalama;
    char cinsiyet;
};

struct ogrenci kisi;
struct ogrenci *ptr; //Pointer tanımlandı (struct türünde veri adresi tutuyor)
ptr=&kisi;

(*ptr).isim="Riceb"; //Pointer içeirği kişi olduğu için kişi.isim riceb oldu.
ptr→isim="Riceb"; //Yukarıdakiyle aynı.
```

```
struct ogrenci{
    char *isim;
    int yas;
    float ortalama;
    char cinsiyet;
};

int main(void){

struct ogrenci riceb;

riceb.isim="Hello";

return 0;
}
```

❗ Pointerlerin içeriğine yukarıdaki gibi tanım yapılabilir ancak dizilere yapmak için strcpy() fonksiyonu kullanılmalıdır.

Struct İçinde Struct

- Struct'ler node'lerle birinine bağlanır ve oklarla birbirlerini gösterirler.

```
#include <stdio.h>

struct dgko{
    int gun,ay,yil;
};

struct ogrenci{
    char *isim;
    int yas;
    float ortalama;
    char cinsiyet;
    struct dgko date;
};

int main(void){
    struct ogrenci recep;
    recep.isim="Recep Kaan Öz";
    recep.date.gun=18;
    //recep.date={18,06,2006};  //BÖYLE BİR KULLANIM YOK.

    return 0;
}
```

Struct'ları Fonksiyon İçinde Kullanma

```
#include <stdio.h>
struct info{
    int sayi;
};

void function(struct info);  //Prototip

int main(void){
    command;
    return 0;
}

void function(struct info x){  //Fonksiyonun kendisi
    command;
}
```

Typedef

- Bir temel type'nin özelliğini başka bir kelimeye atar.

```
typedef int riceb; //riceb kelimesi de int kelimesiyle aynı şekilde kullanılabilir ol
du.
riceb x = 18;

typedef float yejeb;
yejeb y = 1.333333;
```

Typedef Struct:

- typedef struct kullanarak tanım yaparken her tanımda struct yazmamıza gerek kalmaz.

```
typedef struct{
    char isim[15];
    int yas;
    float ortalama;
    char cinsiyet;
}ogrenci;   //yukarıdaki struct'ı typedef'e atadı

ogrenci x; //x ogrenci tipinde bir değişken oldu.
```

Enumerations

- Const komutunda olduğu gibi çalışır ancak struct’lara benzer.

```
enum level{
    FIRST,
    SECOND,
    THIRTH
};
int main(void){
    enum level riceb;
    riceb=SECOND; //1
    return 0;
}
```

- Eğer enum elemanlarına değer atamazsanız ilk elemana 0 verir ve diğerlerine de bir arttırarak devam eder.
- Eğer ilk değere bir sayı verirseniz diğerlerini o sayıyı baz alarak arttırarak ilerler.
- İsterseniz her değere farklı bir sayı verebilirsiniz.

Union

- Birleşik bir yapı kurmak için kullanılır.
- Union'lar struct'ların aksine tek bellek alanında bütün değişkenleri tutar. (En fazla alan kaplayan değişkeninkinde)

```
struct value{
    int x; //4 byte
    int y; //4 byte
    char data[20]; //20 byte
};    //4+4+20=28 byte alan kaplar.

union value{
    int x; //4 byte
    int y; //4 byte
    char data[20]; //20 byte
}    //20 en büyüğü olduğu için toplam 20 byte alan kaplar.
```

- Unionlar aynı anda sadece bir veri tutabilirler.
    - Örneğim isim ve yaş değişkenleri bulunduran bir union'o önce isim sonra yaş girilise sadece yaşı tutar ve önceki veriyi siler.
- Union'a girilen değer her veri tipi ile çağırıla bilir.
    - Aşağıdaki örnekte sadece letter D olmasına rağmen ascii değişkeni yazdırıldığında D harfinin ascii karşılığını verecektir.

```
union value{
    int ascii;
    char letter;
}

int main(void){
    union value data;
    data.letter='D';
```

```
printf("%c",data.letter); //'D'
printf("%d",data.ascii); //68
```